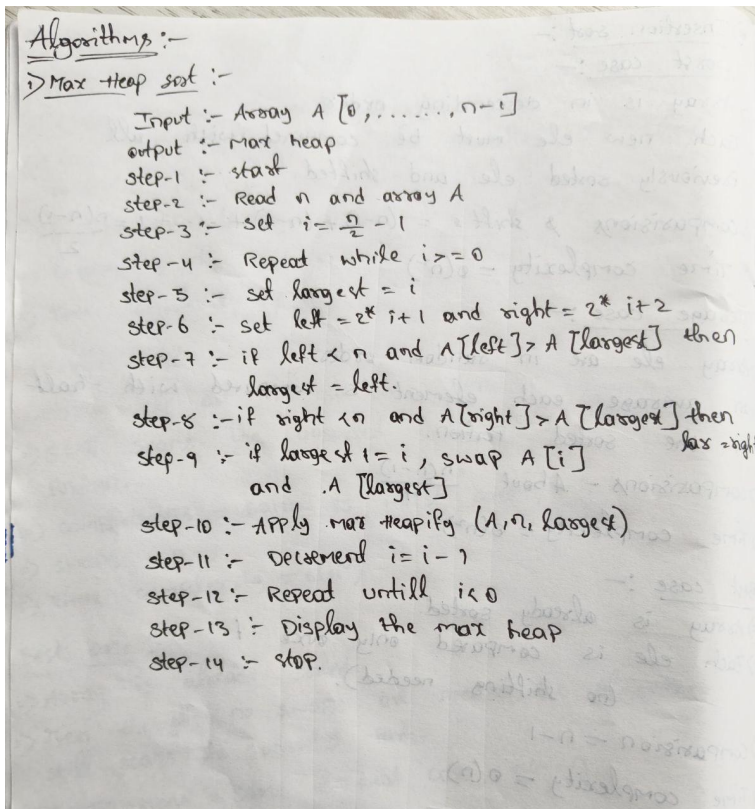


PRACTICAL-03

AIM: Implementation of max-heap sort algorithm

MAX-HEAP SORT

ALGORITHM:



PROGRAM:

```
#include <iostream>
using namespace std;

void MaxHeapify(int arr[], int size, int i) {
    int largest = i;
    int l = 2 * i + 1;
    int r = 2 * i + 2;

    if (l < size && arr[l] > arr[largest])
        largest = l;

    if (r < size && arr[r] > arr[largest])
        largest = r;
```

```
    if (largest != i) {
        swap(arr[i], arr[largest]);
        MaxHeapify(arr, size, largest);
    }
}

void HeapSort(int arr[], int size) {

    for (int i = size / 2 - 1; i >= 0; i--)
        MaxHeapify(arr, size, i);

    for (int i = size - 1; i >= 1; i--) {
        swap(arr[0], arr[i]);
        MaxHeapify(arr, i, 0);
    }
}

int main() {
    int size;
    cout << "Enter number of elements: ";
    cin >> size;

    int arr[size];
    cout << "Enter " << size << " elements:\n";
    for (int i = 0; i < size; i++) {
        cin >> arr[i];
    }

    HeapSort(arr, size);

    cout << "Sorted array: ";
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    return 0;
}
```

OUTPUT:

```
Enter number of elements: 5
Enter 5 elements:
9
0
6
1
7
Sorted array: 0 1 6 7 9
```

TIME COMPLEXITY:

Time complexity :-

```

max heapify (arr[], size, i)  $\rightarrow T(n)$ 
{
    int length = i;  $\rightarrow 1$ 
    int l = (2*i);  $\rightarrow 1$ 
    int r = (2*i)+1;  $\rightarrow 1$ 
    while (l < size && arr[l] > arr[r]) {
        largest = l;  $\rightarrow 1$ 
    }
    if (largest != i) {  $\rightarrow 1$ 
        swap (arr[largest], arr[i]);
        max heapify (largest, size);  $\rightarrow T(n/2)$ 
    }
}

heapsort (i = size/2; i > 1; i--)  $\rightarrow n/2$ 
{
    max heapify (arr, size - i);  $\rightarrow \log n$ 
}

for (i = size; i > 1; i--)  $\rightarrow n-1$ 
{
    swap (arr[i], arr[1]);  $\rightarrow 1$ 
    max heapify (arr, i-1, 1);  $\rightarrow \log n$ 
}

```

$T(n) = T(n/2) + 1$
 using master method
 $T(n) = 1$
 $a=1, b=2, k=0, p=0$
 $\log_b a = \log_2 1 = \frac{\log 1}{\log 2} = 0 < k$
 $p > -1 \Rightarrow 0 > -1$
 $T(n) = O(n^k \log^{p+1} n)$
 $= O(n^0 \log^{0+1} n)$
 $= O(\log n)$
 max heapify $= O(\log n)$
in all cases :-
 > best case $= O(n \log n)$
 > Avg case $= O(n \log n)$
 > worst case $= O(n \log n)$
Build max heap :-
 $T(n) = \frac{n}{2} \times \log n$
 $= O(n \log n)$
heapsort :-
 $T(n) = (n-1) \times \log n$
 $= O(n \log n)$
worst case :-
 $T(n) = T(n/2) + O(1)$
 $= O(\log n)$
 for loop $n-1$ times
 $T(n) = O(n) + (n-1) \times O(\log n)$
 $= O(n \log n)$

Avg case :-
same like the worst case
overall heap sort :- Best case :-
$$T(n) = O(n \log n) + O(n \log n)$$
$$= O(n \log n) \rightarrow \text{best case}$$

CONCLUSION:

The Max-Heap Sort algorithm was successfully implemented using the Max-Heapify method. The largest element is repeatedly placed at the end of the array, resulting in a sorted array in ascending order. The time complexity of Heap Sort is $O(n \log n)$.